

Lecture 02: Source Code to Machine Code

Vivek Kumar

Computer Science and Engineering

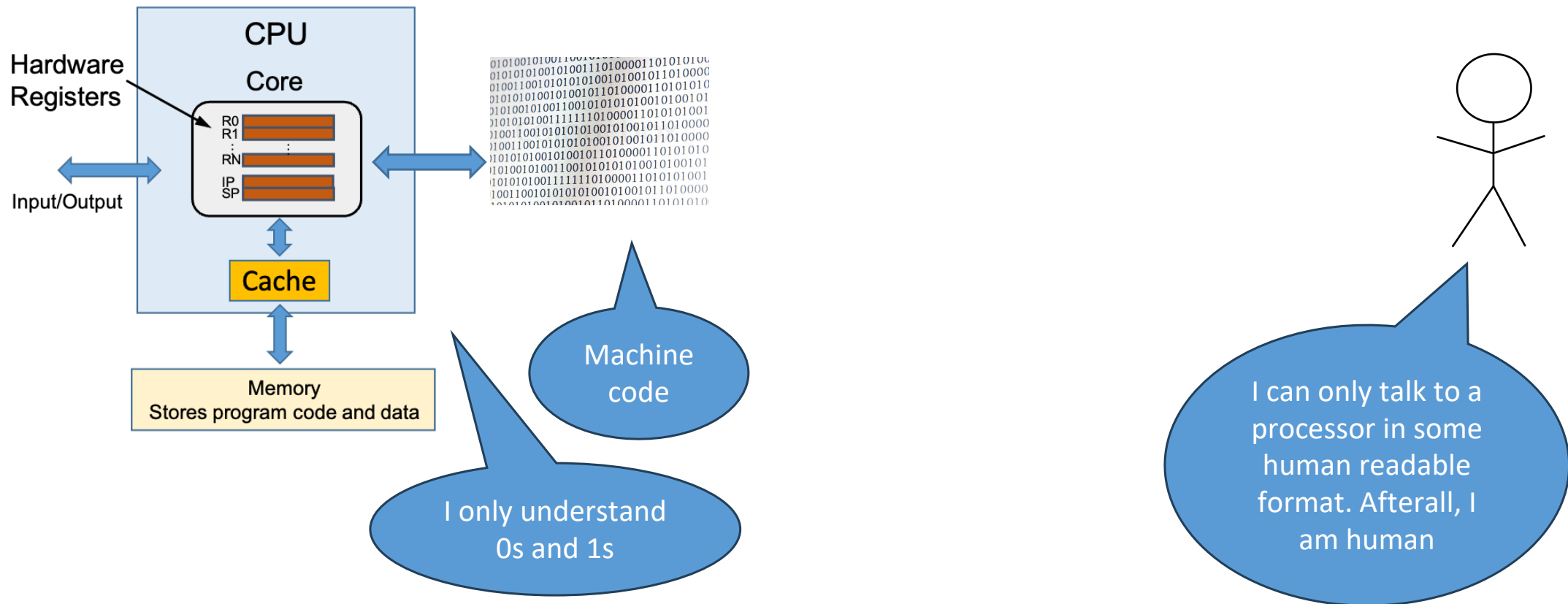
IIIT Delhi

vivekk@iiitd.ac.in

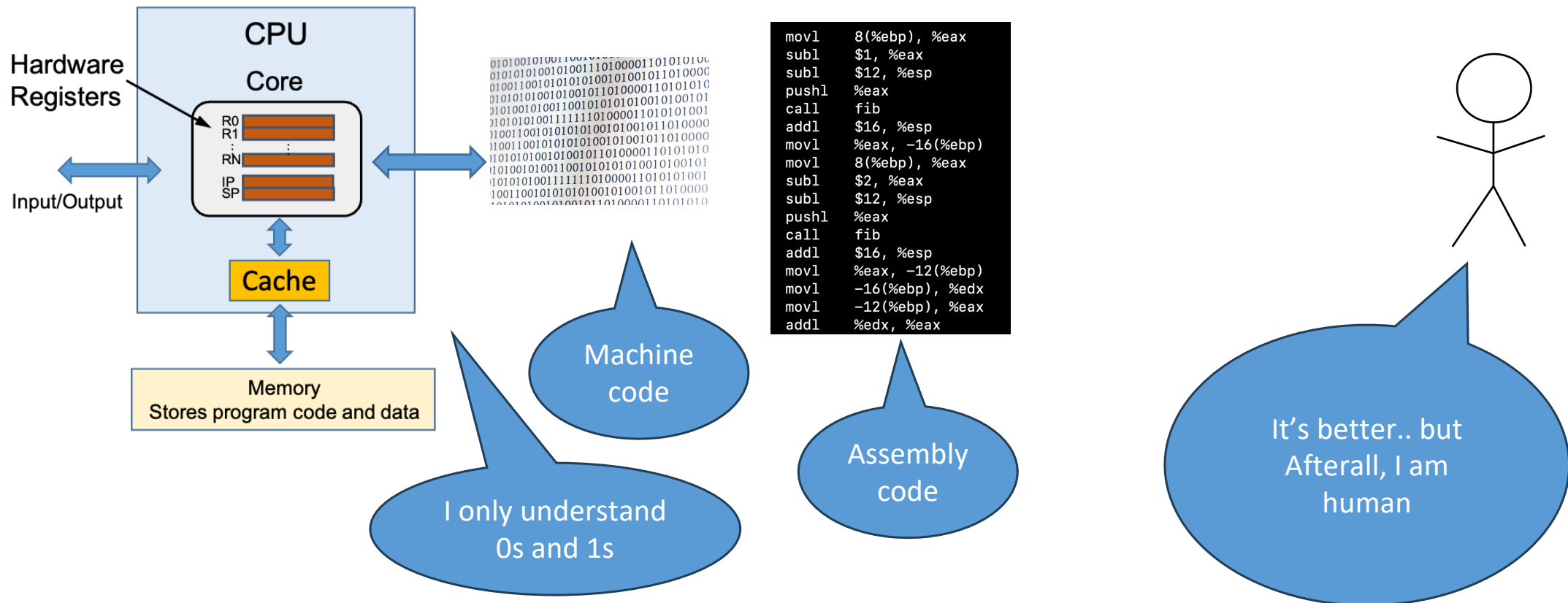
Today's Class

- Compilation steps
- Linking
- Introduction to Executable File Format (ELF)

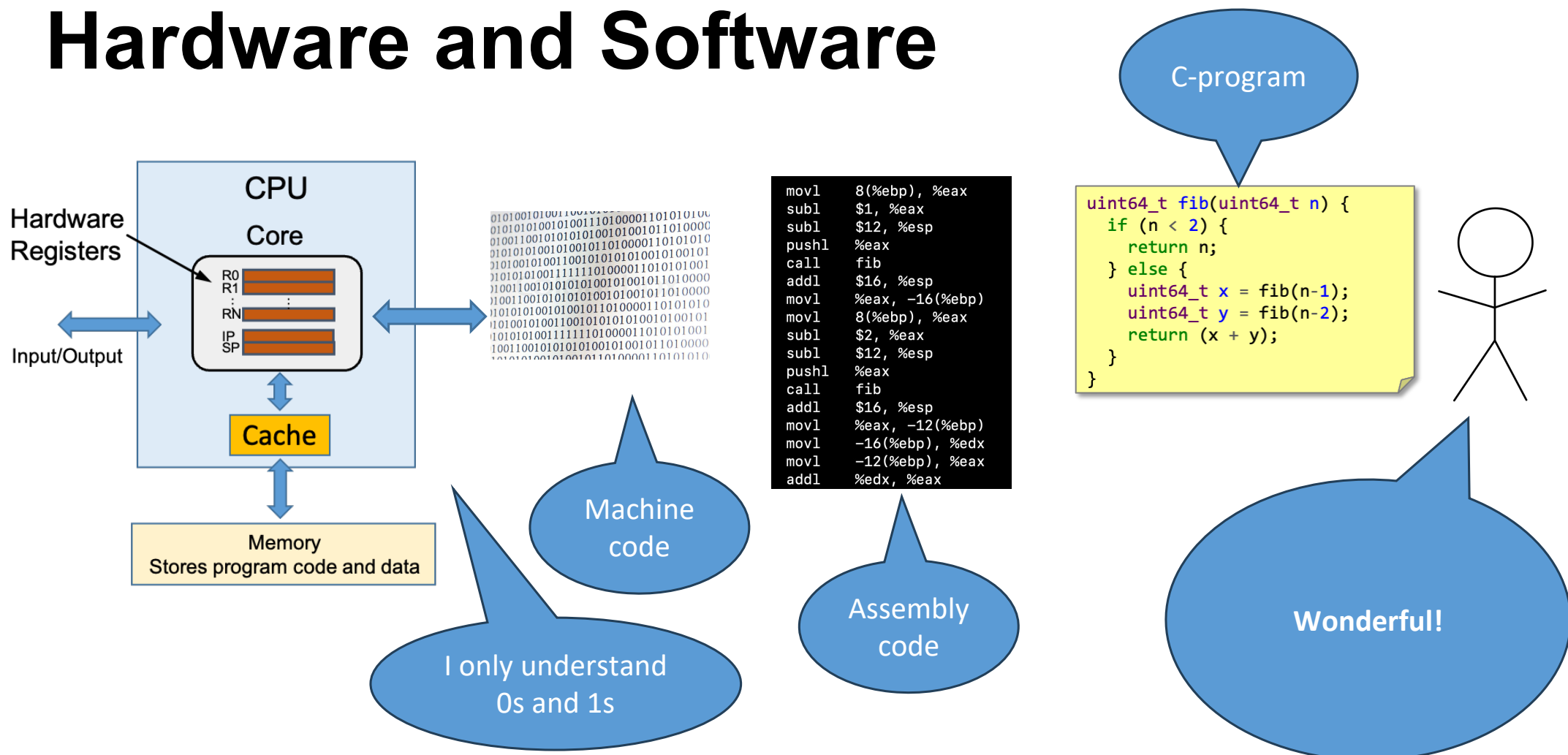
Hardware and Software



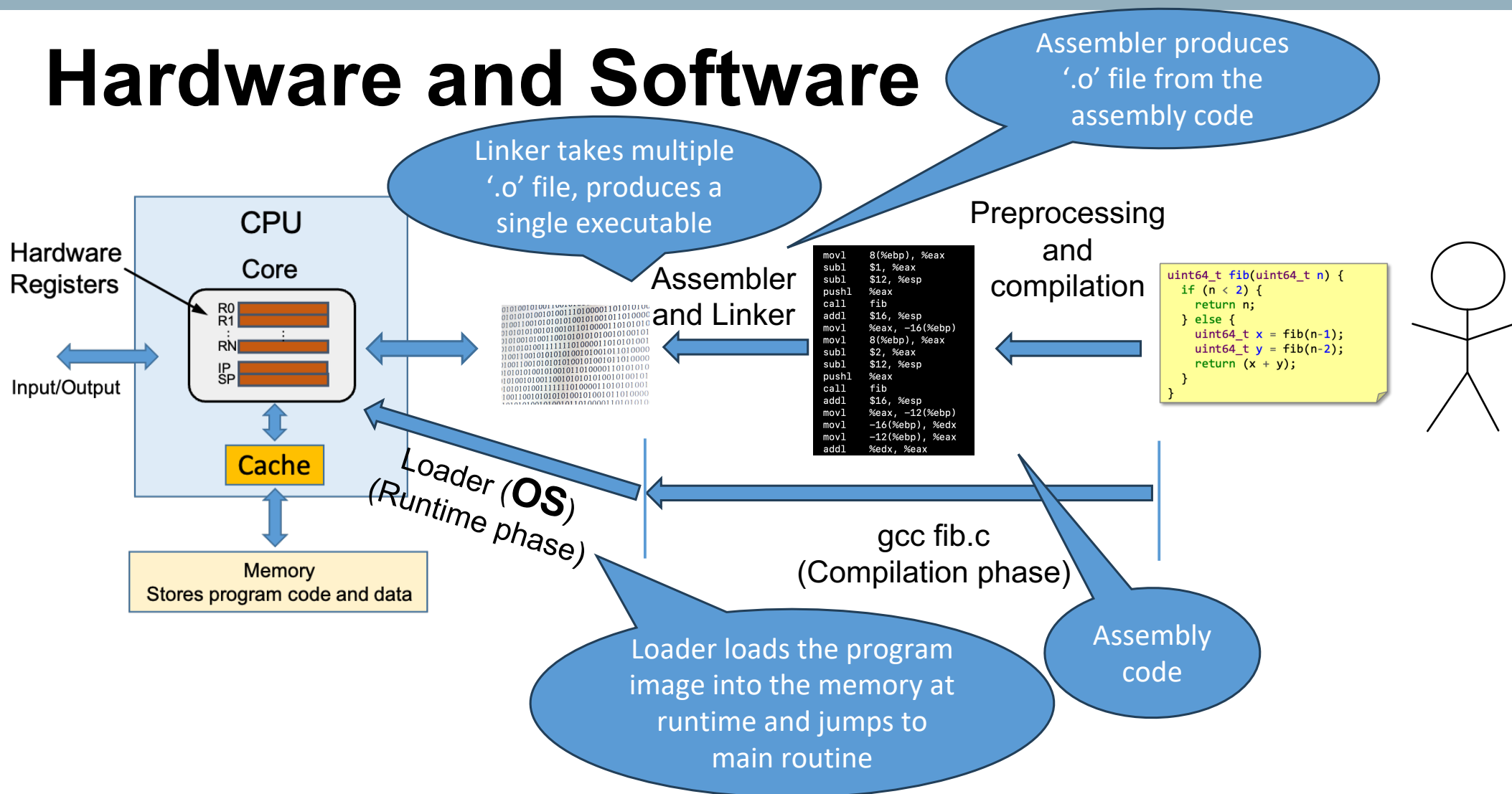
Hardware and Software



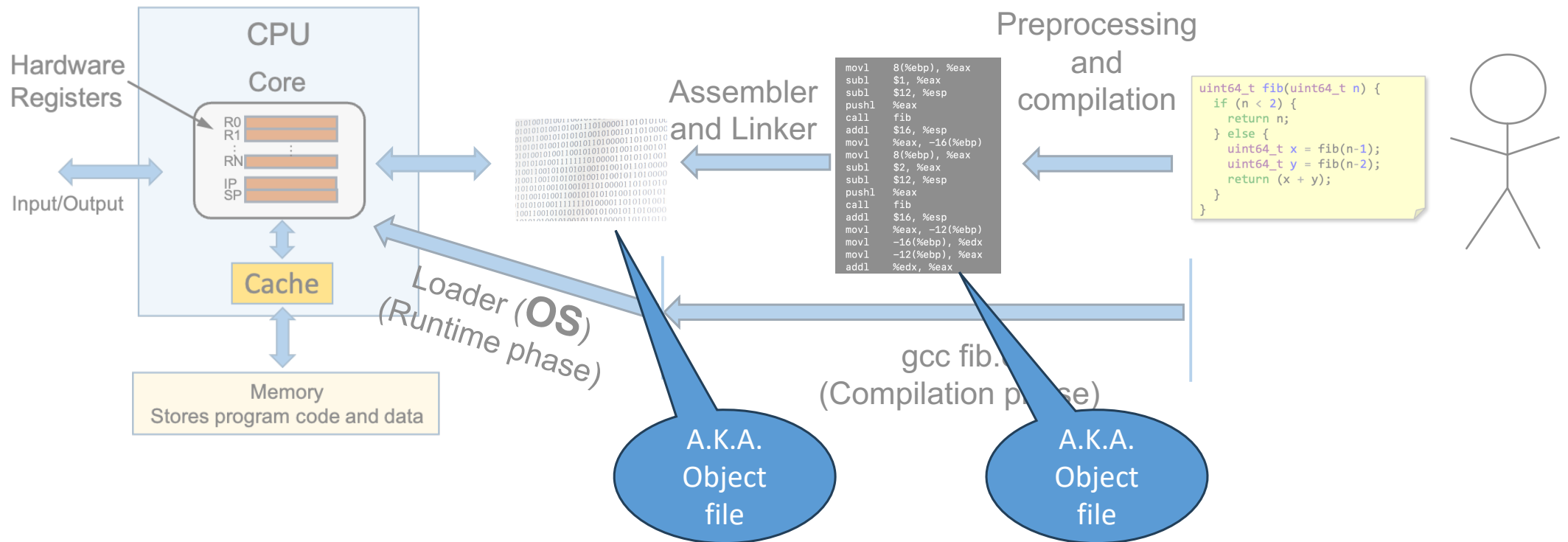
Hardware and Software



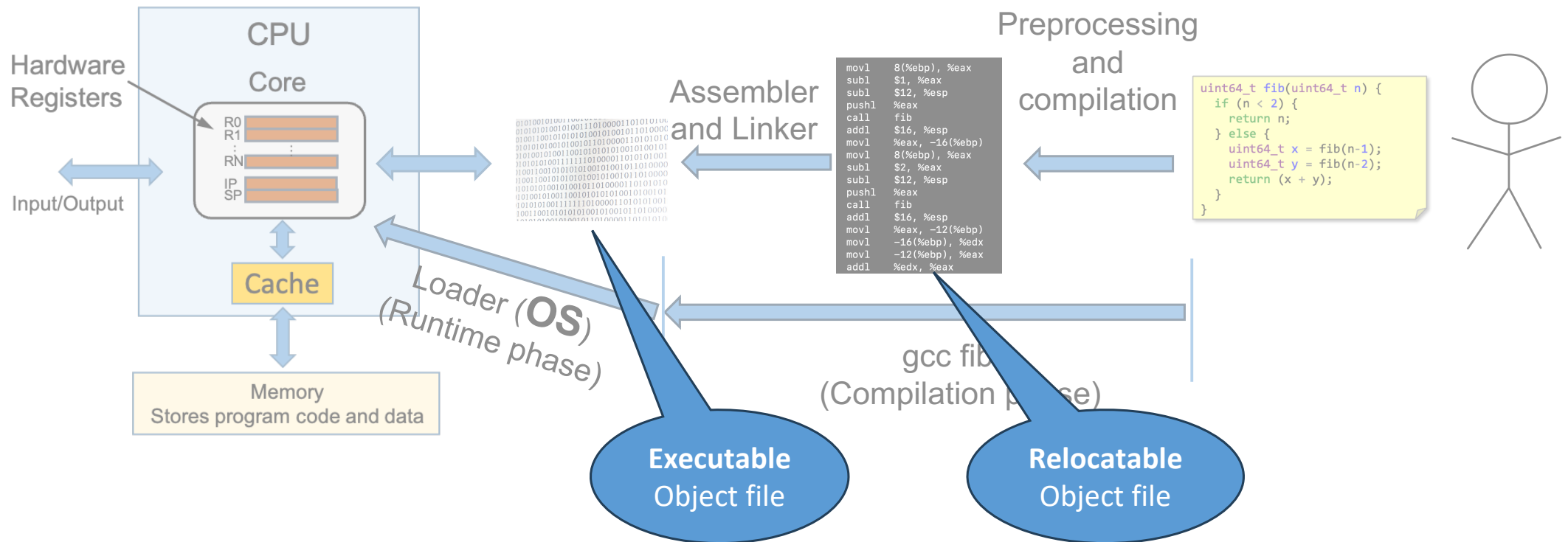
Hardware and Software



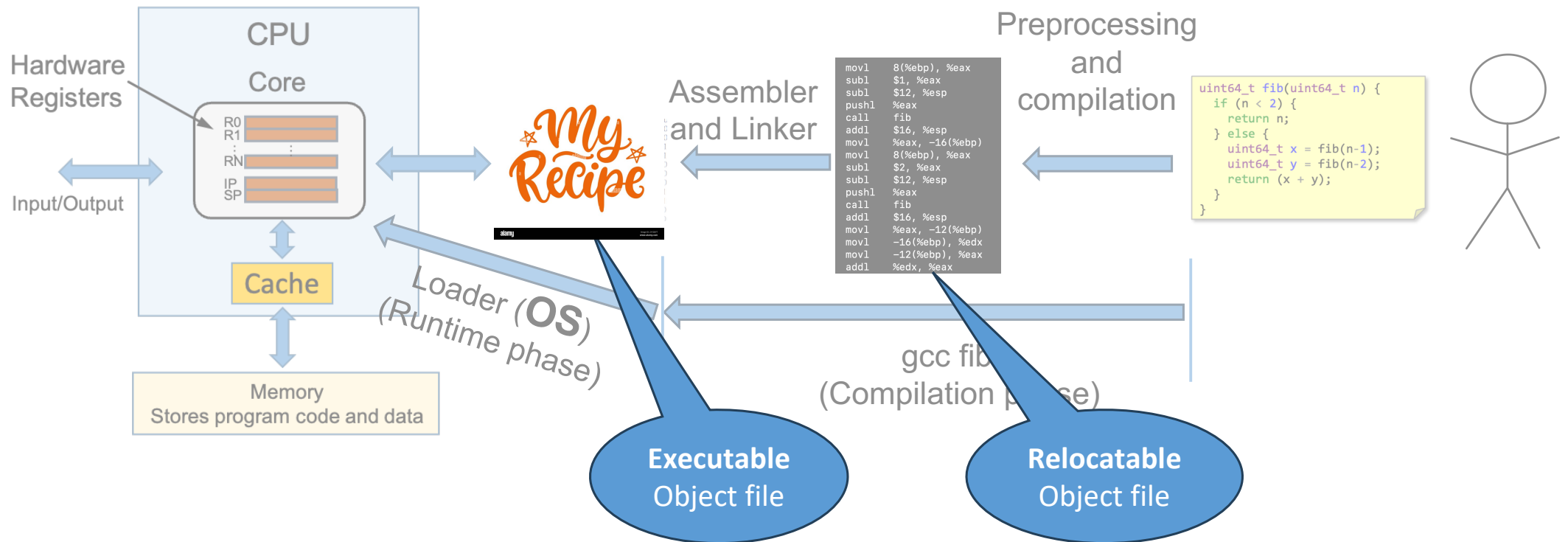
Hardware and Software



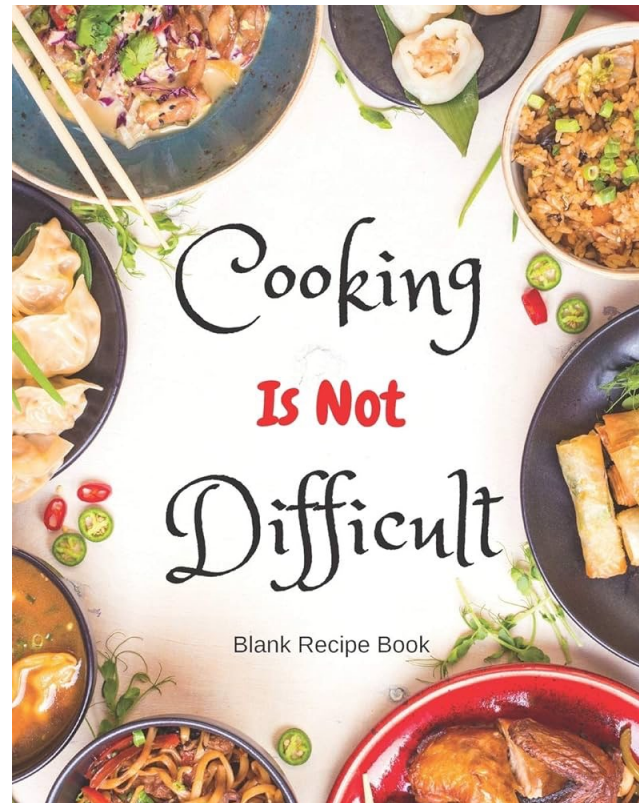
Hardware and Software



Hardware and Software



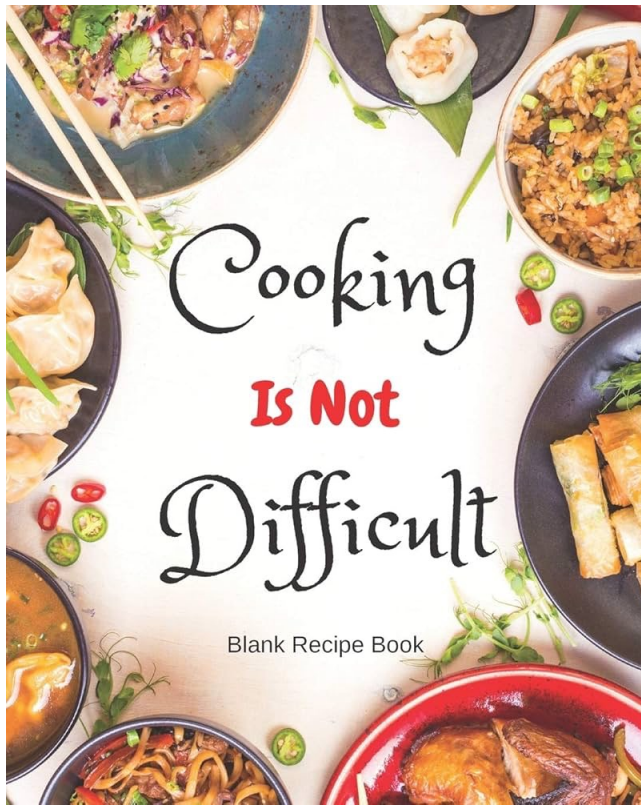
What if you want to cook?



You would need
the **Recipe**

Images source: Google images

Your Recipe book: Will this work?

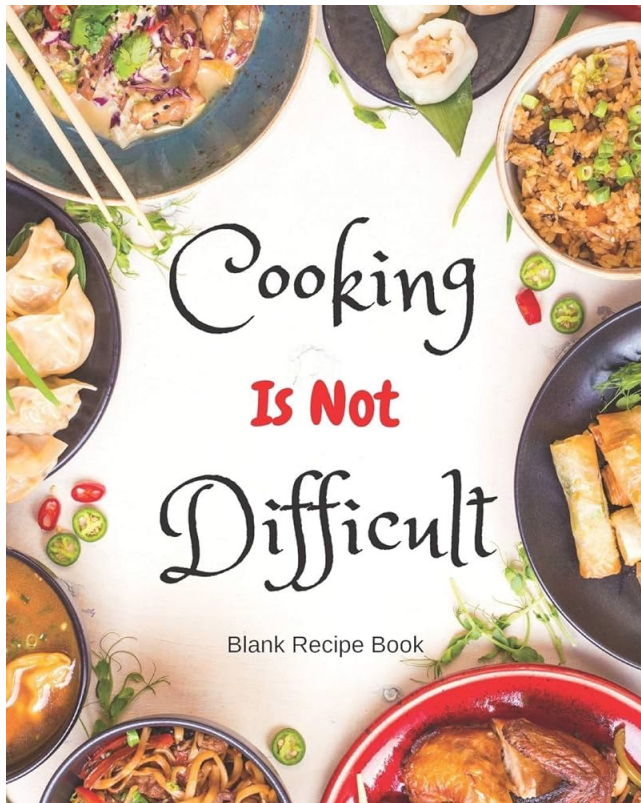


Boil pasta
Cut onion
Prepare dough
Boil potato
Prepare pasta sauce
Prepare pizza sauce
Cut salads
Apply butter on bun

.....

.....

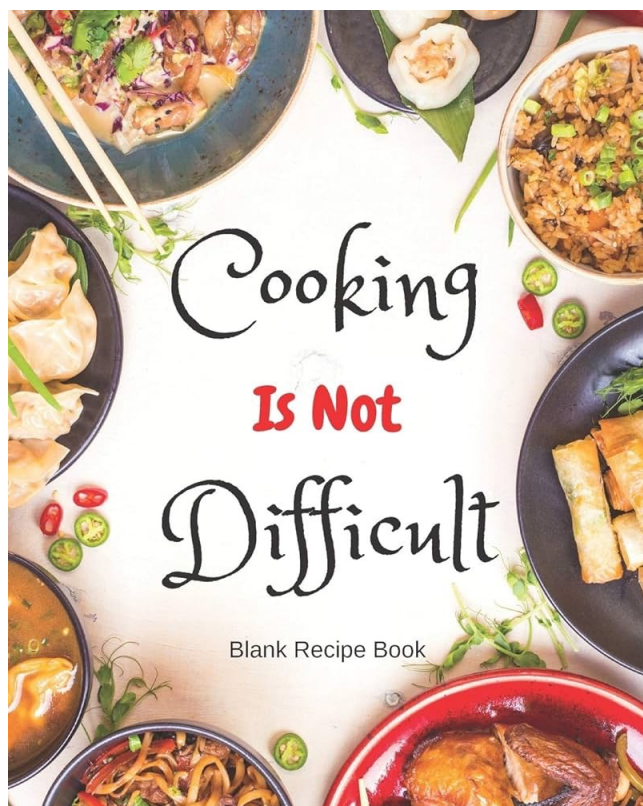
How your Recipe Book Should Look?



Index Page

1. Pasta Pages 1-4
2. Pizza Pages 5-9
3. Burger Pages 10-14

How your Recipe Book Should Look?



Chapter-1: Pasta Recipe

List of ingredients on Page 1: {

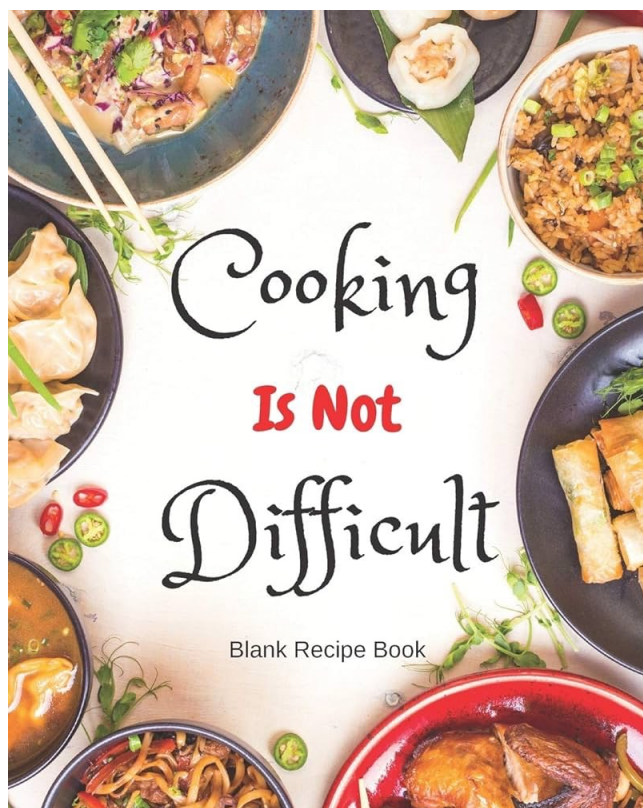
1. Penne
2. Macaroni
3.
4. **Pasta red sauce**
5.

You would need the recipe for this as well. You can either download it and attach to your recipe book, or look online while actually preparing pasta. Any tradeoffs?

}

Step-by-step instructions on Page 2-4

Your Recipe book: Will this work?



- Who is the cook?
 - The one who executes the service
 - E.g., the Operating System (the **loader** to be specific)
- What is the recipe?
 - A well-structured format for recording recipe that the cook can understand
 - E.g., the executable file format
- The index page in the book
 - Header in the executable file format that helps in browsing the book
- The ingredients in the recipe?
 - Program data stored in the executable file format
- The red sauce
 - Statically linked reference if you download its recipe and permanently attach to your recipe book
 - Dynamically linked reference if you look it up online only while actually cooking
- Who prepares the recipe book??
 - Its you who is playing the role of a compiler!

Relocatable Object Files

- Symbols (functions and variables) are not bound to any specific address
 - It will happen after linking
 - Imagine you receiving the recipe for pasta, pizza, and burger from three different people and you haven't linked them together to form a recipe book yet!
 - Pasta.txt, Pizza.txt, and Burger.txt
- Contains code and data suitable for linking with other object files to create the final executable or shared object
 - The final recipe book!

Executable Object Files (**Recipe book!**)

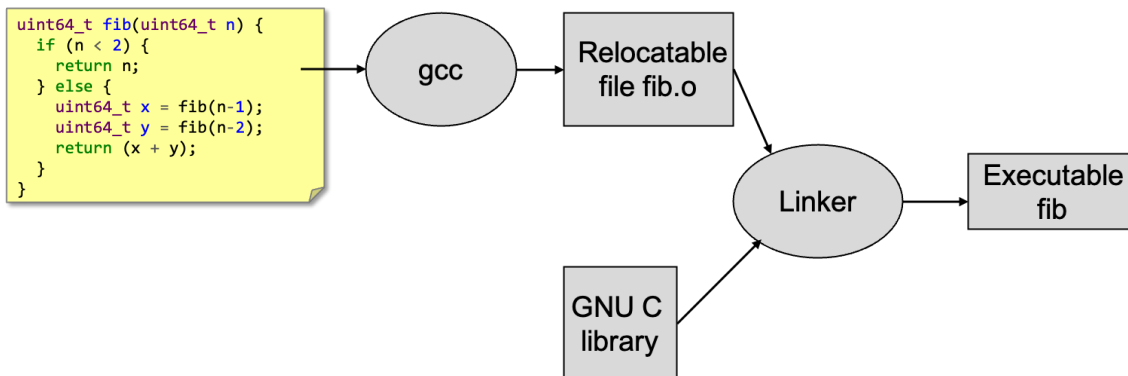
- It is similar to the relocatable file, but all symbols (functions and variables) have addresses resolved
- Contains all the information for the OS loader to run the program

Today's Class

- Compilation steps
- Linking
- Introduction to Executable File Format (ELF)

Linking (Process of Preparing the Recipe book!)

```
$ gcc -o fib fib.c      // compiling + linking
$ file fib
...ELF 64-bit shared object...dynamically linked
$ gcc -c fib.c         // compiling
$ file fib.o           // relocatable file
fib.o
$ gcc -o fib fib.o     // linking
```

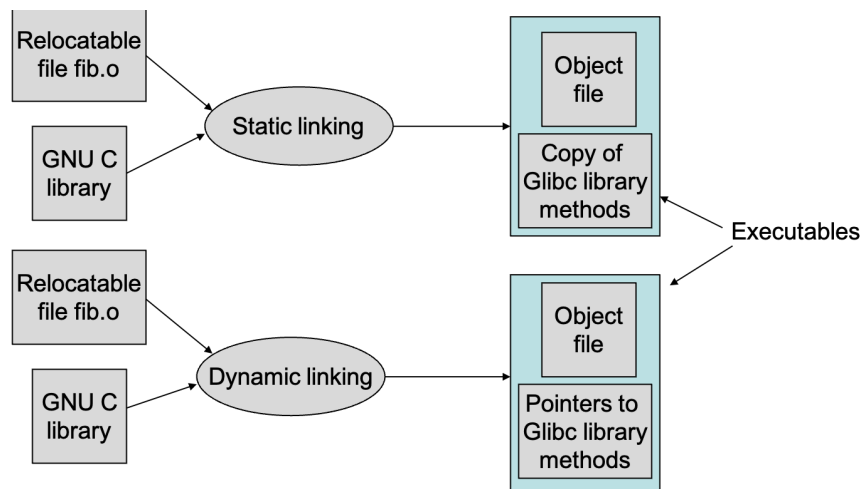


- Combines several relocatable files (.o) into a single executable
 - E.g., Pasta.o, Pizza.o, and Burger.o
- Enable separate compilation of files
 - Changes in one file does not affect other files
 - Recompilation process is small
 - Its possible that you received the recipe of pasta, pizza and burger written from three different persons separately for your recipe book

Static and Dynamic Linking

```
$ gcc -static -o fib fib.c
$ file fib
...ELF 64-bit executable...statically linked
```

```
$ gcc -o fib fib.c
```



● Static linking

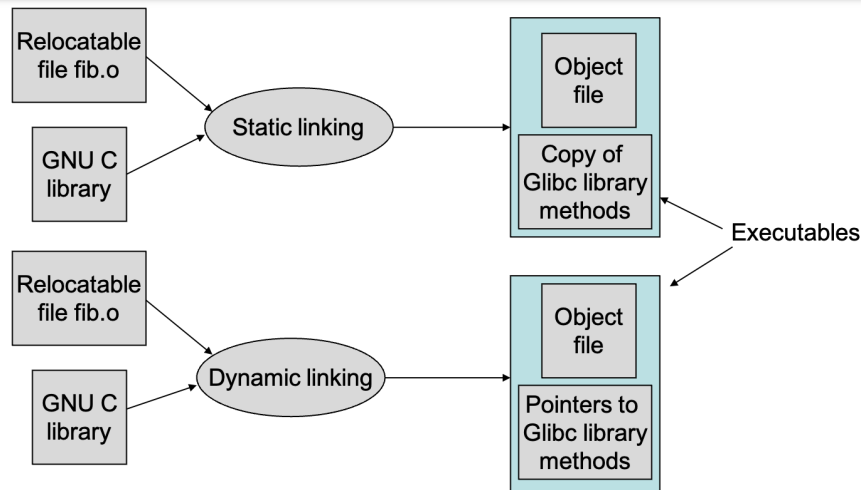
- Each and every library modules referenced in the relocatable file is copied into the final executable
 - Static binding at compile time (e.g., downloading the pasta sauce recipe and attaching it to the recipe book)

● Dynamic linking

- Final executable only contains references (pointers) to the library method instead of the copy of a library method
 - Binding with library done at runtime during execution (e.g., hyperlinking the online recipe of red pasta sauce in the recipe book)

Static and Dynamic Linking

```
$ gcc -static -o fib fib.c
$ file fib
...ELF 64-bit executable...statically linked
$ ls -l fib
-rwxrwxr-x 1 vivek vivek 845304 Aug  6 10:26 fib
$ gcc -o fib fib.c
$ ls -l fib
-rwxrwxr-x 1 vivek vivek 8328 Aug  6 10:27 fib
```



- Static linking
 - Each and every library modules referenced in the relocatable file is copied into the final executable
 - Static binding at compile time (e.g., downloading the pasta sauce recipe and attaching it to the recipe book)
- Dynamic linking
 - Final executable only contains references (pointers) to the library method instead of the copy of a library method
 - Binding with library done at runtime during execution (e.g., hyperlinking the online recipe of red pasta sauce in the recipe book)
- Which is better?
 - A recipe book with a few pages or the one with several pages (of course you always have the internet access while cooking)

Dynamic Linking

- **Advantages**

- Modular programming
 - If library requires changes/recompilation, the executables referring it does not need recompilation
- Saves disk space
 - Executable just contain references
 - Multiple executables can access the same **.so** at runtime

- **Disadvantage**

- What if the shared library changes or is missing at runtime?

Today's Class

- Compilation steps
- Linking
- Introduction to Executable File Format (ELF)
 - [The recipe book!](#)

Executable and Linkable Format (ELF)

- Standard **format** for binary files (object files and core dumps) on Linux based OS
 - Contains all the information required by the OS to run the program
 - Platform independent format
 - A standard format eases the process of dynamic linking, loading and runtime control on a program
- Microsoft Windows OS uses Portable Executable (PE)
- IBM AIX OS uses Extended Common Object File Format (X-COFF)

Binary File Format

- Having a standard format for binary files helps linker and loaders in carrying out its operations easily



I know how
to operate
a “Human”



I know how
to operate
a “Human”

ELF Motivation – Another Example (1/2)

Contents	
1 Introduction	6
2 Chapter Two Title	8
2.1 Section Title	8
2.2 Section Title	9
2.3 Section Title	11
2.3.1 SubSection Title	12
2.3.2 SubSection Title	12
3 Chapter Three Title	13
3.1 Section Title	13
3.1.1 SubSection Title	14
4 Chapter Four Title	15
4.1 Section Title	15
4.2 Section Title	16
4.3 Section Title	16
5 Conclusion	19
A Appendix Title	20

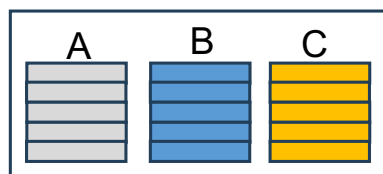
5

Chapter 2	
Chapter Two Title	
2.1 Section Title	
Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.	

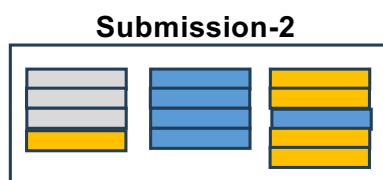
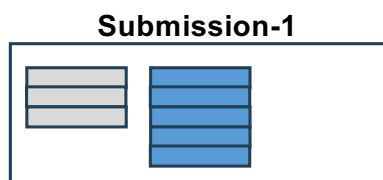
8

- Assume you are doing some research that you have to publish
- You have put a lot of effort, but can you just write as you like?
- There are certain rules/format for writing a research work

ELF Motivation (2/2)



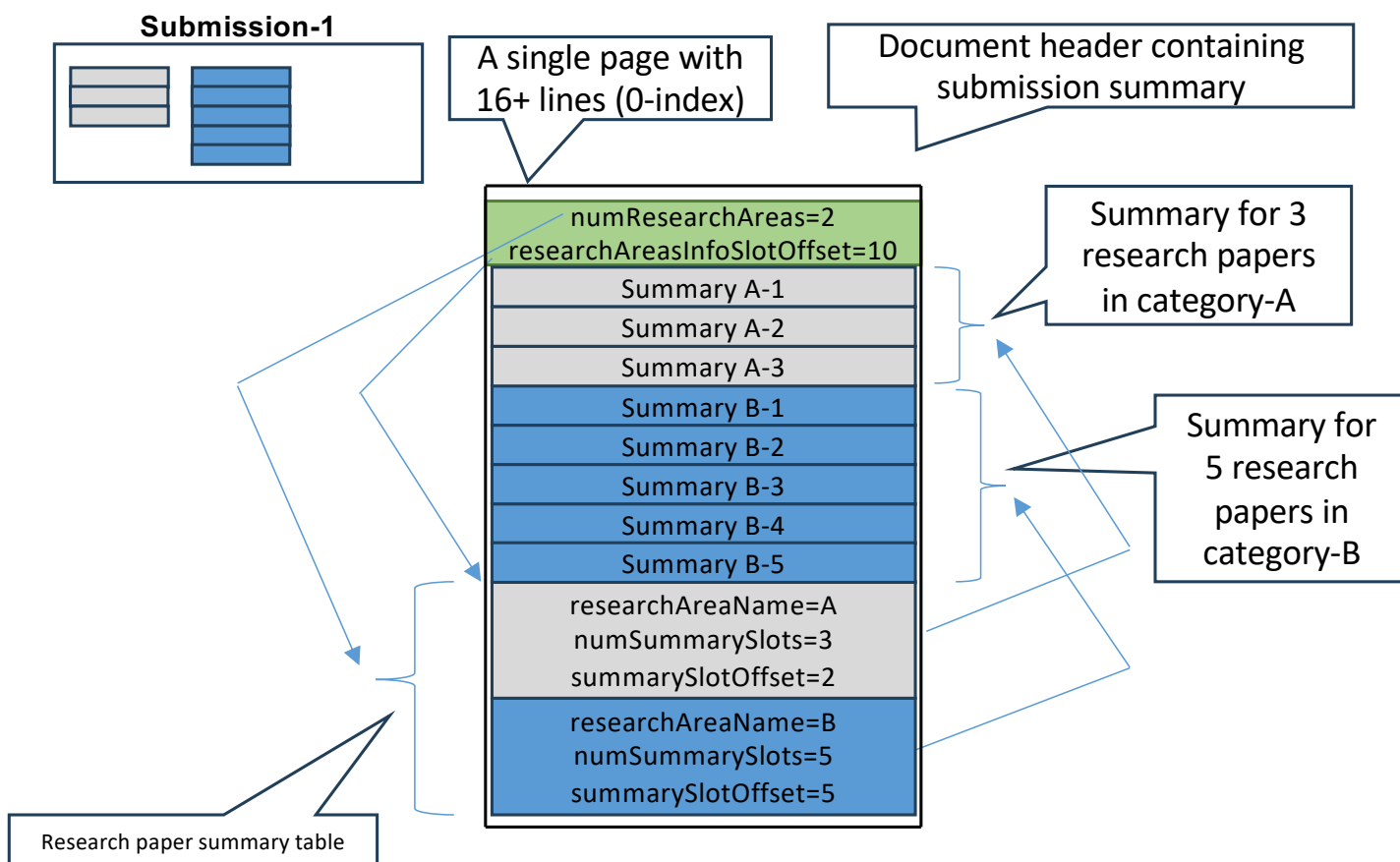
Expectations: You would submit the document in this layout



Reality: Different styles and combinations followed by students

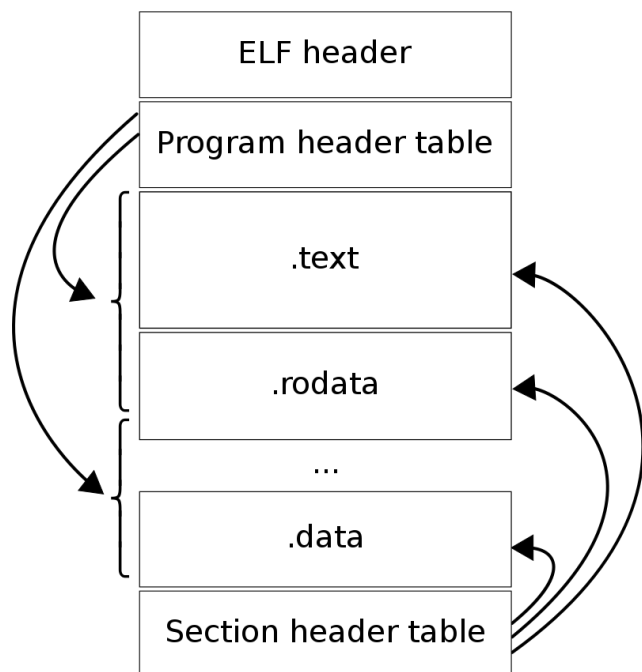
- Assume that you have an assignment to provide a single line summary of 5 articles from 3 different topics A, B and C
 - Total 15 articles (3x5)
 - Yes, ChatGPT does it easily 😊

ELF Motivation (2/2)



- I propose a specific layout to overcome this challenge and to make it easy to understand your document
- I can easily “**load**” the student “**file**” from any number of students in my “**memory**” if they follow a standard “**format**” to store information, e.g., as the one shown below for the **Submission-1** shown here

Executable and Linkable Format (ELF)



Note: For simplicity, we will only discuss ELF format for 32-bit object files

Image source: https://en.wikipedia.org/wiki/Executable_and_Linkable_Format

- **ELF header**
 - Provides a roadmap for the entire file organization
 - Always at offset zero of the object file
 - Provides entry point address for execution
- **Two views of an ELF file**
 - Linkable view (relocatable file)
 - Execution view (executable file, shared object)
- **Linkable view**
 - Section header table
 - One section header for each section
 - Sections
 - Contains data required for linking
 - Machine code, global variables, (initialized and un-initialized), symbol tables, line mapping between machine code and original C code, etc.
- **Execution (or Loader) view**
 - Program header table
 - One program header for each segment
 - Segments
 - Created by merging several sections
 - Contains information required for by the loader for execution
- **Contiguous chunk of memory (ELF header, PHT, SHT, each section, each segment)**

Reading Materials

- Executable and Linkable Format

- <https://www.cs.cmu.edu/afs/cs/academic/class/15213-f00/docs/elf.pdf>
- <https://man7.org/linux/man-pages/man5/elf.5.html>

Next Lecture

- Dissecting an ELF file
 - It will be the topic for your Assignment-1
 - Will be released on 22nd August